Moto-Taxi Rush

Cahier de réalisation d'un jeu vidéo 2D en Java (libGDX) - à destination d'un agent de développement IA

Comment utiliser ce document. Il est écrit pour être lu par une IA qui développe le jeu. Il fixe l'idée, les règles, les chiffres, l'architecture et onze étapes à suivre dans l'ordre. Chaque étape a un livrable et un critère de validation : on ne passe à la suivante que lorsque le critère est rempli.

1. L'idée du jeu

Moto-Taxi Rush est un jeu d'évitement à défilement vertical. Le joueur pilote une moto-taxi (un "benskin") sur une route à trois voies. Les voitures, camions et nids-de-poule arrivent par le haut de l'écran : il faut les éviter. Des passagers qui attendent au bord de la route rapportent des points quand on les ramasse. Plus on avance, plus la route va vite. Une partie dure en général entre une et trois minutes.
Pourquoi ce choix : les règles tiennent en quelques lignes, tout le jeu peut être fait avec des formes colorées (aucun graphisme à créer au début), et il couvre les notions essentielles : boucle de jeu, entrées, collisions, états d'écran, difficulté progressive, sauvegarde du meilleur score.

2. Choix techniques

Élément	Choix	Remarque
Langage	Java 17 ou plus	Le même que le backend habituel du développeur
Framework	libGDX	Gère boucle de jeu, rendu 2D, entrées, sons, sauvegarde
Génération du projet	gdx-liftoff (générateur officiel libGDX)	Crée un projet Gradle multi-modules : core, lwjgl3
Build	Gradle (wrapper commité)	Lancement : ./gradlew lwjgl3:run
IDE	IntelliJ IDEA	Ouvrir le dossier du projet, laisser Gradle importer
Plateforme cible	Desktop (PC) d'abord	Android et web seulement en option, après la v1
Graphismes au départ	Rectangles colorés (ShapeRenderer)	Les vrais sprites viennent à l'étape 9
Sauvegarde	libGDX Preferences	Pour le meilleur score, pas de base de données
Tests	JUnit 5 sur la logique pure	Sans fenêtre ni OpenGL (voir étape 10)

Les versions exactes de libGDX et de Gradle sont celles proposées par gdx-liftoff au moment de la création : l'agent doit les vérifier dans la documentation officielle plutôt que de les deviner.

3. Règles du jeu (numérotées pour pouvoir les citer)

Les valeurs ci-dessous sont des points de départ. Elles doivent vivre dans une classe de constantes unique (`GameConfig`) pour pouvoir être ajustées sans toucher au reste du code.

Réf	Règle	Valeur de départ
R1	L'écran logique est en portrait, avec un viewport fixe	480 x 800 unités (FitViewport)
R2	La route a trois voies de largeur égale	3 voies de 160 unités

Réf	Règle	Valeur de départ
R3	La moto occupe une seule voie à la fois et change de voie d'un cran (gauche ou droite)	Moto 60 x 100, en bas de l'écran
R4	On ne peut pas sortir de la route : aller à gauche sur la voie 1 ne fait rien	-
R5	Les obstacles apparaissent en haut, sur une voie aléatoire, et descendent	Voitures, camions, nids-de-poule
R6	Vitesse de défilement : augmente avec le temps, plafonnée	250 + 12 par seconde, maximum 700 unités/s
R7	Intervalle entre deux apparitions : diminue avec le temps, plancher	De 1,2 s vers 0,45 s
R8	Deux obstacles ne doivent jamais bloquer les trois voies en même temps : une voie libre au minimum	-
R9	Toucher un obstacle coûte une vie et déclenche 1,5 s d'invincibilité (la moto clignote)	3 vies
R10	Un passager apparaît à la place d'un obstacle une fois sur quatre ; le ramasser rapporte des points	+50 points
R11	Un passager non ramassé qui sort de l'écran est simplement perdu, sans pénalité	-
R12	Survivre rapporte des points en continu	+10 points par seconde
R13	La partie se termine à 0 vie ; le meilleur score est sauvegardé s'il est battu	-

4. Contrôles

- **Clavier :** flèches gauche et droite, ou touches Q et D (ou A et D). Échap : pause. Entrée : valider dans les menus.
- **Souris ou tactile (optionnel) :** toucher la moitié gauche de l'écran change de voie vers la gauche, la moitié droite vers la droite.

5. Architecture du code

Séparer la **logique du jeu** (règles, positions, score) de l'**affichage** (rendu, sons). C'est ce qui rend les tests possibles et le code lisible.

```
core/src/main/java/cm/motorush/
  MotoRushGame.java        Classe principale (extends Game), crée les écrans
  config/GameConfig.java   Toutes les constantes (R1 à R13)
  screen/                  MenuScreen, PlayScreen, PauseScreen, GameOverScreen
  model/                   Moto, Obstacle, Passager, ObstacleType (données + règles)
  logic/                   Spawner, DifficultyCurve, CollisionChecker, ScoreKeeper
  render/                  Renderer (ShapeRenderer puis SpriteBatch)
  storage/                 HighScoreStore (Preferences)
core/src/test/java/cm/motorush/   Tests JUnit sur logic/ et model/
```

- **logic/** ne dépend jamais d'OpenGL ni des classes graphiques : elle prend un temps écoulé (delta) et renvoie un état. C'est la partie testée.
- **screen/** gère les entrées et orchestre logic + render. Aucune règle de jeu à cet endroit.
- Les objets du jeu utilisent des rectangles (`com.badlogic.gdx.math.Rectangle`) pour les collisions.

6. Les onze étapes de réalisation

Pour chaque étape : faire le travail, vérifier le critère de validation en lançant le jeu, faire un commit (message qui dit ce qui a été fait, avec le numéro de l'étape), puis passer à la suivante. Ne jamais cumuler deux étapes dans un commit.

Étape 0 - Préparation

À faire : Vérifier Java 17+, créer le projet avec gdx-liftoff (modules core et lwjgl3, nom de package cm.motorush), initialiser Git, poser un .gitignore adapté (build/, .gradle/, .idea/, *.class, assets générés), commiter le wrapper Gradle.
Validation : Le projet s'ouvre dans IntelliJ et ./gradlew lwjgl3:run affiche la fenêtre par défaut de libGDX.

Étape 1 - Fenêtre et boucle de jeu

À faire : Remplacer l'exemple par une classe MotoRushGame et un PlayScreen vide. Mettre en place la caméra orthographique et le FitViewport 480 x 800 (R1). Fond gris foncé, titre de fenêtre "Moto-Taxi Rush", taille de fenêtre portrait. Gérer le redimensionnement.
Validation : La fenêtre s'ouvre en portrait, se redimensionne sans déformer le contenu, et se ferme proprement.

Étape 2 - La route et la moto

À faire : Dessiner la route (3 voies séparées par des pointillés qui défilent) et la moto (rectangle coloré) sur la voie centrale. Gérer le changement de voie au clavier (R3, R4), avec un petit glissement animé plutôt qu'une téléportation.
Validation : Les flèches déplacent la moto d'une voie à la fois, sans jamais sortir de la route.

Étape 3 - Obstacles et apparitions

À faire : Créer Obstacle et Spawner. Les obstacles apparaissent en haut sur une voie aléatoire et descendent à la vitesse de défilement (R5). Supprimer ceux qui sortent de l'écran (pas de fuite mémoire). Garantir une voie libre (R8).
Validation : Des obstacles de tailles différentes descendent en continu ; jamais trois voies bloquées à la fois ; le nombre d'objets en mémoire reste stable après une minute de jeu.

Étape 4 - Collisions et vies

À faire : Implémenter CollisionChecker (rectangles). Une collision retire une vie et active l'invincibilité de 1,5 s avec clignotement (R9). Afficher les vies à l'écran.
Validation : Toucher un obstacle retire exactement une vie, pas plusieurs d'un coup ; pendant l'invincibilité on traverse les obstacles sans perdre de vie.

Étape 5 - Passagers et score

À faire : Ajouter les passagers (R10, R11) et ScoreKeeper : points au fil du temps (R12) et par passager. Afficher le score.
Validation : Ramasser un passager ajoute 50 points et le fait disparaître ; le score augmente de 10 par seconde.

Étape 6 - Difficulté progressive

À faire : Créer DifficultyCurve : vitesse (R6) et intervalle d'apparition (R7) en fonction du temps de jeu, avec plafond et plancher. Aucune valeur en dur dans le code : tout vient de GameConfig.
Validation : Après deux minutes la route est visiblement plus rapide qu'au début, sans devenir impossible ; les limites sont respectées.

Étape 7 - États et écrans

À faire : Ajouter MenuScreen (Jouer, Quitter), PauseScreen (touche Échap), GameOverScreen (score, rejouer, retour au menu). La fin de partie à 0 vie (R13) mène au Game Over. Rejouer remet tout à zéro proprement.
Validation : On enchaîne menu, partie, pause, game over, nouvelle partie, plusieurs fois de suite, sans reste de l'ancienne partie (score, obstacles, vies).

Étape 8 - Meilleur score

À faire : Créer HighScoreStore avec libGDX Preferences. Sauvegarder le meilleur score s'il est battu, l'afficher au menu et au Game Over.
Validation : Après fermeture et relance du jeu, le meilleur score est toujours là.

Étape 9 - Sons et graphismes

À faire : Remplacer les rectangles par des sprites libres de droits (par exemple Kenney.nl ou OpenGameArt : vérifier et noter la licence de chaque fichier dans un fichier ASSETS.md). Ajouter musique, bruit de collision et de passager, avec un bouton pour couper le son. Charger les ressources une seule fois (AssetManager ou équivalent) et les libérer (dispose).
Validation : Le jeu est plaisant à regarder et à écouter, les licences sont documentées, aucun fichier d'asset sans origine connue.

Étape 10 - Tests, packaging, documentation

À faire : Écrire des tests JUnit sur la logique pure : (a) DifficultyCurve respecte le plafond et le plancher, (b) le Spawner ne bloque jamais les trois voies, (c) l'invincibilité empêche de perdre une vie. Produire un JAR exécutable (tâche de distribution du module lwjgl3). Rédiger un README : présentation, captures d'écran, commandes de lancement, contrôles.
Validation : ./gradlew test passe sur un clone vierge ; le JAR se lance par double clic ou java -jar ; le README suffit à un inconnu pour jouer.

7. Extension optionnelle : classement en ligne

À ne faire qu'**après** l'étape 10, comme projet séparé qui réutilise les compétences Spring Boot. Une petite API REST (Java, Spring Boot, base PostgreSQL ou H2) avec deux opérations : envoyer un score (pseudo + score) et lire le top 10. Le jeu appelle l'API en fin de partie et affiche le classement. Points d'attention : validation des entrées, format d'erreur JSON uniforme, aucun secret dans le dépôt, et un jeu qui reste jouable hors ligne si l'API est injoignable. Note honnête : un classement envoyé par le client peut être truqué ; c'est acceptable pour un jeu d'apprentissage.

8. Conventions de travail

- Un commit par étape au minimum, avec un message clair ("Etape 4 : collisions et vies (R9)"). Pas de gros commit unique à la fin.
- Ne jamais commiter de fichiers générés (build/, .gradle/, .idea/, JAR de sortie) ni de secrets.
- Toutes les valeurs de réglage dans GameConfig. Aucune valeur magique dispersée.
- Classes courtes et nommées pour ce qu'elles font. Commentaires seulement là où le code ne se suffit pas.
- En cas de doute sur une API libGDX, consulter la documentation officielle plutôt que d'inventer une méthode.

9. Pièges à éviter

Piège	Conséquence	Bonne pratique
Mouvement lié au nombre d'images par seconde	Le jeu va plus vite sur un PC rapide	Toujours multiplier par le delta (Gdx.graphics.getDeltaTime())
Créer des objets (Rectangle, etc.) à chaque image	Ralentissements par ramasse-miettes	Réutiliser les objets ou utiliser un pool
Oublier dispose()	Fuite mémoire, plantage après plusieurs parties	Libérer textures, sons, SpriteBatch, ShapeRenderer
Rendre la logique dépendante du rendu	Impossible à tester	Garder logic/ sans import graphique
Obstacles supprimés pendant le parcours de la liste	Plantage ou saut d'éléments	Parcourir à l'envers ou utiliser un itérateur
Tout faire d'un coup	Un bug impossible à localiser	Suivre les étapes, tester à chaque fois

10. Définition de "terminé" (version 1.0)

- Les étapes 0 à 10 sont faites et commitées, chacune avec son critère de validation rempli.
- Une partie complète est jouable : menu, jeu, pause, game over, rejouer.
- Les règles R1 à R13 sont toutes respectées (vérifier une par une).
- ./gradlew test passe ; le jeu démarre depuis un clone vierge avec les commandes du README.
- Aucun fichier généré ni secret dans le dépôt ; licences des assets documentées.